ADVANCED CS PRACTICUM | DISTRIBUTED BLOCKCHAIN TAKEAWAY ASSIGNMENT

MiniChain Network: Build, Synchronize, Attack, and Validate

Name: Yashank | **Roll Number:** t26049

1. System Architecture & Multi-Process Node Isolation

The MiniChain Network is implemented as a decentralized peer-to-peer replicated ledger operating across three autonomous operating system processes launched in separate command terminals: **node1 (localhost:5001)**, **node2 (localhost:5002)**, and **node3 (localhost:5003)**. In strict fulfilment of the independence requirement, the three processes share no in-memory Python objects and no common disk file. Each node maintains its own isolated in-memory Blockchain instance and persists its local state exclusively to a node-specific JSON ledger file (state_node1.json, state_node2.json, and state_node3.json) so that stopping and restarting a node preserves its local chain history prior to going offline.

Module / Class	Implemented Interface Methods	Core Architectural & Functional Responsibilities

blockchain.py
class Block
	`calculate_hash()`
	`mine(difficulty=3)`
	`to_dict(), from_dict()`
		Encapsulates block fields (index, timestamp, transactions, previous_hash, nonce, miner_id, hash). Computes deterministic SHA-256 digests and increments nonce during Proof-of-Work mining until hash begins with '000'.

blockchain.py
class Blockchain
	`create_genesis_block()`
	`add_block(block)`
	`validate_new_block()`
	`is_valid()`
	`validate_chain_list()`
		Initializes the deterministic Genesis block (B0), enforces index continuity and previous_hash linkage prior to appending any block, and performs full-chain cryptographic audits from Genesis to tip during synchronization.

node.py
class BlockchainNode
	`mine_block(transactions)`
	`validate_block(block)`
	`broadcast_block(block)`
	`receive_block(block)`
	`sync_chain()`
	`handle_peer_request()`
		Manages the multi-threaded TCP listener socket, dispatches JSON messages with a 2.0s timeout so offline peers never crash active nodes, implements the interactive CLI (chain, mine, validate, peers, sync, tamper, exit), and enforces longest-valid-chain selection.

2. Cryptographic Block Specification & TCP Communication Protocol

Deterministic Hashing & Proof-of-Work (Difficulty = 3): Every block binds its header and transaction payload into a single SHA-256 digest computed over the canonical pipe-delimited string: **`hash = SHA256(index || timestamp || transactions || previous_hash || nonce || miner_id)`**, where transactions are serialized via key-sorted JSON. A mined block is valid only if its hexadecimal SHA-256 digest begins with three leading zeroes (000…). To ensure all three isolated processes start with the exact same Genesis block (B0) without pre-shared files, create_genesis_block() uses fixed parameters (index=0, timestamp='0', transactions=['GENESIS'], previous_hash='0'*64, miner_id='system'), converging deterministically at nonce = 1199 with hash 000606778da36492f4abc3e4eb75f7a0b8d53989e3807c33e6e4a479c02c77d7.

Message Type	Direction	Serialized JSON Payload	Receiving Node Validation & Consensus Handling
NEW_BLOCK	Node -> Peers	`{"type": "NEW_BLOCK", "sender": id, "block": {...}}`	Broadcast upon local mining. Receiving peer verifies block.index == latest.index + 1, recalculated SHA-256 hash, previous_hash == latest.hash, and '000' PoW prefix before appending.
GET_CHAIN	Node -> Peer	`{"type": "GET_CHAIN", "sender": id}`	Issued during sync across all configured peers over TCP (2.0s socket timeout). Unreachable or stopped peers are logged cleanly without interrupting node execution.
CHAIN	Peer -> Node	`{"type": "CHAIN", "sender": id, "chain": [...]}`	Returns the peer's complete serialized blockchain. Requester validates every block from B0 to tip via validate_chain_list() and adopts the candidate chain only if it is valid and longer (or repairs local corruption).

3. Converged Blockchain Ledger State (B0 -> B1 -> B2 -> B3)

Following the complete execution of propagation, tampering recovery, offline catch-up, and invalid-chain rejection tests (T1–T9), all three independent nodes (node1, node2, and node3) converged on the identical 4-block valid chain summarized below:

Block	Miner	Timestamp	Transactions	Nonce	Previous Block SHA-256 Hash	Mined Block SHA-256 Hash
B0	system	0	["GENESIS"]	1199	000000000000000000000000...	000606778da36492f4abc3e4...
B1	node1	1793531290	["A->B:10"]	14407	000606778da36492f4abc3e4...	000e10228d9c5d5cbbb74395...
B2	node2	1793531366	["B->C:5"]	10862	000e10228d9c5d5cbbb74395...	000e40424f394d658980f6cc...
B3	node1	1793531499	["C->D:3"]	930	000e40424f394d658980f6cc...	00089b8ef98e0ccbf7d303b3...

4. Test Case Verification Matrix (T1 – T9)

Test	Operation Executed	Expected Distributed Outcome	Actual Verified Result & Console Evidence
T1	Start node1 (5001), node2 (5002), and node3 (5003) in separate terminals.	Three independent node processes run; each begins with the same Genesis block B0.	PASS: All three nodes initialized on ports 5001–5003 with identical Genesis hash 000606778d... and logged Chain summary: B0(000606).
T2	On node1: mine A->B:10	node1 mines Block 1 and broadcasts it; node2 and node3 validate and append Block 1.	PASS: node1 mined B1 (000e10228d..., nonce 14407) and broadcast to node2, node3; both logged Hash valid, Previous hash valid, Proof-of-work valid, Block appended.
T3	On node2: mine B->C:5	Block 2 propagates; all three nodes converge on identical B0 -> B1 -> B2 chains.	PASS: node2 mined B2 (000e40424f..., nonce 10862) and broadcast to node1, node3; all three nodes converged on B0(000606) -> B1(000e10) -> B2(000e40).
T4	On node2: tamper 1 0 A->B:1000, then run validate across nodes.	node2 reports INVALID. node1 and node3 remain VALID and unchanged.	PASS: node2 logged Tampered Block 1 Tx 0: 'A->B:10' -> 'A->B:1000' and Local chain INVALID after tamper. node1 and node3 both logged Local chain VALID.
T5	On node2: restore/sync via sync and run validate.	node2 obtains a valid chain from peers and returns to the same chain as node1/node3.	PASS: node2 logged Local height: 2 (Valid: False), verified Peer (node1) height: 2 and Peer (node3) height: 2 as valid, logged Local chain replaced and Local chain VALID.
T6	Stop node3 (exit). On node1: mine C->D:3.	node1 and node2 reach B3; node3 remains one block behind at B2 while offline.	PASS: node1 mined B3 (00089b8ef9..., nonce 930), node2 appended B3, and node1 logged Peer node3 (localhost:5003) unreachable during broadcast.
T7	Restart node3 on port 5003 and run sync.	node3 fetches a longer valid chain from peers and reaches the same B3 tip.	PASS: node3 restarted at Local height: 2 (Valid: True), verified Peer (node1) height: 3 and Peer (node2) height: 3 (Candidate chain valid), and logged Local chain replaced -> B3(00089b).
T8	Present a longer invalid chain: node2 injects 2 invalid blocks (height 5); node1 runs sync.	The receiving node (node1) rejects the peer chain even though it is longer (height 5 > 3).	PASS: node1 (at Local height: 3) inspected Peer (node2) height: 5, logged Candidate chain from node2 INVALID - rejected, and logged Local chain retained.
T9	Present a longer valid chain: reset node1 to B0 (height 0) and run sync against height-3 peers.	The receiving node validates the candidate chain and adopts it under the longest-valid-chain rule.	PASS: node1 (at Local height: 0) validated height-3 chains from node2 and node3 (Candidate chain valid) and logged Local chain replaced.
After T7–T9 (Final Convergence)		B0 -> B1 -> B2 -> B3 (Height 3, VALID)	B0 -> B1 -> B2 -> B3 (Height 3, VALID)	B0 -> B1 -> B2 -> B3 (Height 3, VALID)

5. Recorded Console Evidence (output/node1.txt, output/node2.txt, output/node3.txt)

output/node1.txt (Port 5001)

```
[node1] Started on port 5001 with Genesis 000606778d...
[node1] Mining Block 1...
[node1] Block mined: 000e10228d...
[node1] Broadcasting Block 1 to node2, node3
[node1] Block 2 received from node2
[node1] Hash valid | Prev valid | PoW valid
[node1] Block appended
[node1] Chain: B0(000606)->B1(000e10)->B2(000e40)
```

output/node2.txt (Port 5002)

```
[node2] Started on port 5002 with Genesis 000606778d...
[node2] Block 1 received from node1
[node2] Hash valid | Prev valid | PoW valid
[node2] Block appended
[node2] Mining Block 2...
[node2] Block mined: 000e40424f...
[node2] Broadcasting Block 2 to node1, node3
[node2] Tampered Block 1 Tx 0: 'A->B:10'->'A->B:1000'
```

output/node3.txt (Port 5003)

```
[node3] Started on port 5003 with Genesis 000606778d...
[node3] Chain summary: B0(000606)
[node3] Block 1 received from node1
[node3] Hash valid | Prev valid | PoW valid
[node3] Block appended
[node3] Block 2 received from node2
[node3] Hash valid | Prev valid | PoW valid
[node3] Block appended
[node3] Chain: B0(000606)->B1(000e10)-
```

```
[node1] Local chain VALID
[node1] Mining Block 3...
[node1] Block mined: 00089b8ef9...
[node1] Peer node3 (localhost:5003) unreachable
[node1] Local height: 3 (Valid: True)
[node1] Peer (node2) height: 5
[node1] Candidate chain from node2 INVALID - rejected
[node1] Local height: 0 (Valid: True)
[node1] Peer (node2) height: 3 -> Candidate valid
[node1] Local chain replaced
```

```
[node2] Local chain INVALID after tamper
[node2] Local height: 2 (Valid: False)
[node2] Peer (node1) height: 2 -> Candidate valid
[node2] Local chain replaced | Local chain VALID
[node2] Block 3 received from node1 -> Appended
[node2] Injected 2 invalid blocks for T8 (Height: 5)
[node2] Local height: 5 (Valid: False)
[node2] Peer (node1) height: 3 -> Candidate valid
[node2] Local chain replaced
[node2] Chain: B0->B1->B2->B3(00089b)
```

```
>B2(000e40)
[node3] Local chain VALID
[node3] Stopping node...
[node3] Started on port 5003 with Genesis 000606778d...
[node3] Local height: 2 (Valid: True)
[node3] Peer (node1) height: 3
[node3] Candidate chain valid
[node3] Peer (node2) height: 3 -> Candidate valid
[node3] Local chain replaced
[node3] Chain: B0->B1->B2->B3(00089b)
```

6. Technical Analysis & Distributed Consensus Evaluation

6.1 Cryptographic Immutability & Local Tamper Isolation (Tests T4 & T5): In Test T4, executing 'tamper 1 0 A->B:1000' on node2 mutated Block 1's transaction list in memory and in state_node2.json without re-mining. When validate ran, Blockchain.validate_chain_list() recomputed B1.calculate_hash() over the modified payload, yielding a SHA-256 digest that neither matched B1's stored hash (000e10228d9c5d5c...) nor satisfied Block 2's previous_hash pointer. Even if an attacker recalculated B1's hash and cascaded the update into B2.previous_hash, both blocks would lose their '000' Proof-of-Work prefix unless the attacker re-mined every subsequent block faster than the honest network. Because node1 and node3 run in isolated OS processes with separate memory spaces, node2's local corruption had zero impact on their VALID state. In Test T5, running 'sync' on node2 detected that its local chain was invalid (Valid: False), fetched the height-2 chains from node1 and node3 via GET_CHAIN, verified their cryptographic integrity from B0 to B2, and replaced the corrupted local ledger.

6.2 Byzantine Resistance — 'Longest Chain' vs. 'Longest Valid Chain' (Tests T8 & T9): A naive consensus rule that adopts whichever peer advertises the highest block count is trivially vulnerable to Sybil and forgery attacks, as a malicious node can append arbitrary unmined blocks in microseconds. In Test T8, node2 appended two forged blocks with fabricated hashes to reach height 5 while honest peers stood at height 3. When node1 executed sync_chain(), it received node2's 6-block array (height 5) and invoked Blockchain.validate_chain_list(candidate_chain, difficulty=3) prior to comparing chain lengths. Validation immediately flagged the mismatch between the forged blocks' declared hashes, their recalculated SHA-256 digests, and the '000' Proof-of-Work target, causing node1 to reject the height-5 chain ('Candidate chain from node2 INVALID - rejected') and retain its valid height-3 ledger. Conversely, in Test T9, when node1 was reset to Genesis (height 0) and queried peers holding a valid height-3 chain, full cryptographic verification succeeded and node1 adopted B0 -> B1 -> B2 -> B3 under the rule: if len(peer_chain) > len(local_chain) and validate_chain(peer_chain).

6.3 Crash Fault Tolerance & Asynchronous Partition Recovery (Tests T6 & T7): Real-world peer-to-peer networks must tolerate sudden node departures without stalling consensus. In Test T6, node3 was terminated via 'exit' while at height 2 (B0 -> B1 -> B2). When node1 mined Block 3 ('C->D:3', hash 00089b8ef9..., nonce 930), broadcast_block() delivered B3 to node2 and caught the TCP connection refusal on localhost:5003 using a 2.0-second socket timeout, logging 'Peer node3 (localhost:5003) unreachable during broadcast' without raising an unhandled exception. When node3 restarted in Test T7, it reloaded its persisted height-2 state from state_node3.json, queried its peers via sync, verified that the candidate height-3 chain anchored to the identical Genesis block B0(000606778d...) and satisfied all Proof-of-Work links, and atomically updated its local ledger to B3.